

**LAPORAN HASIL PENGUJIAN LOAD BALANCING
ANALISIS PERBANDINGAN ALGORITMA ROUND ROBIN DAN
LEAST CONNECTIONS**

Dosen Pengampu : RUNAL REZKIAWAN, S.Kom.,M.T



Nama : Rizki Amalia Rasyid Ridha

Nim : 105841121223

Kelas : RPL 6 A

**PROGRAM STUDI INFOMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR**

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum untuk mata kuliah *Scalable Systems Design* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk dokumentasi dan analisis komprehensif terhadap uji coba arsitektur *load balancing* menggunakan metode Round Robin dan Least Connections yang diimplementasikan di atas platform Docker Container dan Python. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan, arahan, dan ilmu yang telah diberikan oleh Bapak Runal Rezkiawan, S.Kom., M.T selaku dosen pengampu.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat, menambah wawasan, dan menjadi referensi yang berguna bagi pembaca, khususnya rekan-rekan mahasiswa Informatika.

Makassar, 30 Mei 2026

Rizki Amalia Rasyid

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI	3
BAB I PENDAHULUAN.....	4
A. Latar Belakang	4
B. Rumusan Masalah	4
C. Tujuan Praktikum.....	5
BAB III LANDASAN TEORI.....	6
A. Scalable Systems Design & Load Balancing	6
B. Algoritma Round Robin.....	6
C. Algoritma Least Connections.....	6
D. Docker, Docker Compose, dan Python Flask	6
BAB III METODOLOGI DAN PERANCANGAN SISTEM.....	8
A. Spesifikasi Lingkungan Pengujian.....	8
B. Arsitektur dan Topologi Kontainer	8
C. Implementasi dan Kode Program.....	8
D. Metode Pengujian (<i>Testing Methodology</i>)	11
BAB IV HASIL DAN PEMBAHASAN.....	13
A. Hasil Pengujian Sistem	13
B. Analisis dan Pembahasan.....	17
BAB V KESIMPULAN	19

BAB I

PENDAHULUAN

A. Latar Belakang

Dalam konsep Perancangan Sistem Skalabel (*Scalable Systems Design*), kemampuan sistem untuk menangani pertumbuhan beban kerja tanpa mengorbankan performa adalah hal yang mutlak. Ketika sebuah aplikasi web mengalami lonjakan pengguna secara drastis, satu buah *instance* server tunggal (*single point of failure*) berpotensi besar mengalami kelebihan beban kerja (*resource exhaustion*) yang berujung pada lambatnya waktu respons hingga kegagalan total sistem (*downtime*).

Salah satu strategi paling efektif untuk mencapai skalabilitas horizontal adalah dengan menerapkan *Load Balancing*. Alat penyeimbang beban (*Load Balancer*) bertindak sebagai pintu gerbang utama yang menerima semua lalu lintas permintaan (*traffic request*) dari klien, untuk kemudian mendistribusikannya ke beberapa cluster server backend. Namun, efisiensi dari sistem ini sangat bergantung pada pemilihan algoritma penjadwalan beban yang tepat. Oleh karena itu, praktikum ini dilakukan untuk membandingkan performa dua algoritma fundamental, yaitu Round Robin dan Least Connections, menggunakan simulasi berbasis lingkungan kontainer Docker.

B. Rumusan Masalah

1. Bagaimana cara mengonfigurasi dan mengimplementasikan arsitektur *load balancing* menggunakan Nginx dan Python di dalam ekosistem Docker Compose?
2. Bagaimana karakteristik distribusi beban kerja dari algoritma Round Robin ketika menghadapi variasi waktu respons (*delay*) pada server?

3. Bagaimana keunggulan algoritma Least Connections dalam menangani skenario beban kerja dinamis dibandingkan dengan Round Robin?

C. Tujuan Praktikum

1. Membangun infrastruktur sistem web yang skalabel secara horizontal memanfaatkan Docker, Nginx, dan Flask (Python).
2. Menganalisis secara visual dan logis perilaku distribusi *request* dari masing-masing algoritma berdasarkan pembuktian data log kontainer.
3. Mengkorelasikan hasil pengujian dengan teori perancangan sistem terdistribusi guna menentukan rekomendasi arsitektur terbaik.

BAB III

LANDASAN TEORI

A. Scalable Systems Design & Load Balancing

Skalabilitas arsitektur terbagi menjadi dua jenis: Vertikal (*Scale-up* dengan meningkatkan spesifikasi *hardware*) dan Horizontal (*Scale-out* dengan menambah jumlah mesin server). *Load Balancing* merupakan komponen inti dari skalabilitas horizontal. Penyeimbang beban memastikan tidak ada satu pun server yang bekerja terlalu keras (*overloaded*) sementara server lain menganggur (*idle*), sehingga memaksimalkan nilai *Throughput* dan meminimalkan *Response Time*.

B. Algoritma Round Robin

Algoritma Round Robin bekerja dengan prinsip rotasi melingkar yang sederhana. Setiap permintaan baru dari klien akan diarahkan ke server berikutnya di dalam daftar urutan secara bergiliran. Algoritma ini bersifat *stateless* dan tidak memedulikan metrik performa aktual server seperti penggunaan CPU, memori, ataupun jumlah koneksi yang sedang ditangani.

C. Algoritma Least Connections

Algoritma Least Connections bersifat lebih dinamis dan sensitif terhadap keadaan internal server (*stateful*). Sebelum mengalihkan *request*, *Load Balancer* akan mengecek metrik koneksi aktif pada setiap node backend. Permintaan baru akan selalu diprioritaskan menuju server yang memiliki jumlah koneksi aktif paling sedikit. Metode ini sangat ideal untuk menangani sesi transaksi yang membutuhkan waktu eksekusi berbeda-beda.

D. Docker, Docker Compose, dan Python Flask

Docker memungkinkan pengembang mengemas aplikasi beserta dependensinya ke dalam unit terisolasi yang disebut kontainer. Docker Compose mempermudah proses

definisi dan eksekusi multi-kontainer melalui satu file konfigurasi (docker-compose.yml). Sementara itu, Flask merupakan *micro-framework* berbasis Python yang efisien untuk membangun aplikasi web atau API yang ringan untuk kebutuhan simulasi backend.

BAB III

METODOLOGI DAN PERANCANGAN SISTEM

A. Spesifikasi Lingkungan Pengujian

Pengujian disimulasikan sepenuhnya pada satu buah mesin lokal dengan spesifikasi sebagai berikut:

- a) Sistem Operasi: Windows 11 64-bit
- b) Platform Eksekusi: Docker Desktop Engine v24+
- c) Web Server Engine: Nginx Image nginx:alpine
- d) Application Server: Python Image python:3.9-slim dengan dependensi Flask
- e) Jalur Komunikasi: HTTP Protokol melalui Port Lokal 8085

B. Arsitektur dan Topologi Kontainer

Sistem terdiri dari 4 kontainer terisolasi yang saling terhubung dalam satu jaringan virtual internal Docker:

- 1. lc-roundrobin-nginx-lb-1: Berperan sebagai pintu masuk utama yang meneruskan *traffic* luar ke jaringan internal.
- 2. lc-roundrobin-web-1, web-2, web-3: Tiga replika server Python Flask independen yang menjalankan aplikasi web yang sama namun memiliki identitas Container ID yang unik.

C. Implementasi dan Kode Program

- 1. Konfigurasi Lingkungan Kontainer (docker-compose.yml)

File docker-compose.yml berfungsi sebagai konseptor utama untuk membangun dan menghubungkan seluruh kontainer ke dalam satu jaringan

lokal virtual (*bridge network*). Berikut adalah isi berkas konfigurasi yang digunakan:

```
docker-compose.yml > ...
docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec:json) |
1  services:
2    >Run Service
3    nginx-lb:
4      image: nginx:alpine
5      ports:
6        - "8085:80"
7      volumes:
8        - ./nginx.conf:/etc/nginx/nginx.conf:ro
9      depends_on:
10        - web-1
11        - web-2
12        - web-3
13
14    >Run Service
15    web-1:
16      image: python:3.9-slim
17      container_name: web-1
18      volumes:
19        - ./app
20      working_dir: /app
21      command: sh -c "pip install flask && python app.py"
22
23    >Run Service
24    web-2:
25      image: python:3.9-slim
26      container_name: web-2
27      volumes:
28        - ./app
29      working_dir: /app
30      command: sh -c "pip install flask && python app.py"
31
32    >Run Service
33    web-3:
34      image: python:3.9-slim
35      container_name: web-3
36      volumes:
37        - ./app
38      working_dir: /app
39      command: sh -c "pip install flask && python app.py"
```

2. Pengembangan Aplikasi Backend (app.py)

Setiap server *backend* menjalankan aplikasi web sederhana menggunakan Flask. Di dalam skrip ini, disisipkan fungsi penundaan waktu (*delay/sleep*) konstan sebesar **0.1 detik** menggunakan modul time untuk mensimulasikan proses komputasi riil di dalam server.

```

app.py > home
1 from flask import Flask
2 import os
3 import time
4 import random
5 import socket
6
7 app = Flask(__name__)
8
9 @app.route('/')
10 def home():
11     ip_kontainer = socket.gethostbyname(socket.gethostname())
12     container_id = socket.gethostname()
13
14     if ip_kontainer.endswith('.2'):
15         nama_web = "WEB SERVER 1 (web-1)"
16         warna_aksen = "#4facfe"
17     elif ip_kontainer.endswith('.3'):
18         nama_web = "WEB SERVER 2 (web-2)"
19         warna_aksen = "#00f2fe"
20     elif ip_kontainer.endswith('.4'):
21         nama_web = "WEB SERVER 3 (web-3)"
22         warna_aksen = "#00ff00"
23     else:
24         nama_web = f"WEB SERVER TEMPLATE ({container_id})"
25         warna_aksen = "#ff9a9e"
26
27     waktu_proses = 0.1
28     time.sleep(waktu_proses)
29
30     html_template = f"""
31     <!DOCTYPE html>
32     <html lang="en">
33     <head>
34         <meta charset="UTF-8">
35         <meta name="viewport" content="width=device-width, initial-scale=1.0">
36         <title>Scalable Systems Design - Monitor</title>
37     </head>
38     <body>{{
39         font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
40         background: linear-gradient(135deg, #141e30 0%, #243b55 100%);
41         color: #fff;
42         display: flex;
43         justify-content: center;
44         align-items: center;
45         height: 100vh;

```

3. Konfigurasi Algoritma Nginx (nginx.conf)

File ini memuat instruksi inti bagaimana Nginx mengarahkan lalu lintas data. Pengujian dilakukan dengan memanipulasi baris di dalam blok upstream: Hilangkan tanda '#' untuk menguji Least Connections

```

nginx.conf
1  events { worker_connections 1024; }
2
3  http {
4      upstream opsi_server {
5          #least_conn;
6          server web-1:5000;
7          server web-2:5000;
8          server web-3:5000;
9      }
10
11     server {
12         listen 80;
13         location / {
14             proxy_pass http://opsi_server;
15         }
16     }
17 }

```

D. Metode Pengujian (*Testing Methodology*)

Metode pengujian dilakukan menggunakan pendekatan Automated Load Testing melalui skrip eksternal Python bernama tes.py. Skrip ini memanfaatkan pustaka concurrent.futures dengan modul ThreadPoolExecutor untuk mengirimkan 5 paket data secara serentak dalam milidetik yang sama (*simultaneous multi-threaded requests*).

Langkah-langkah operasional pengujian dijabarkan sebagai berikut:

1. Tahap Persiapan: Menyalakan seluruh kluster server menggunakan perintah terminal:

docker-compose up --build

2. Pengujian Skenario I (Round Robin): Memastikan baris least_conn; pada nginx.conf dalam kondisi mati (diberi tanda komentar). Skrip tes.py dijalankan untuk mencatat bagaimana sifat alokasi beban beruntun bekerja.
3. Pengujian Skenario II (Least Connections): Mengaktifkan direktif least_conn; pada nginx.conf, melakukan *restart* kontainer dengan perintah docker-compose down dilanjutkan docker-compose up --build, lalu

mengeksekusi kembali skrip tes.py untuk mengamati perubahan pola distribusi ketika server mendeteksi adanya lonjakan koneksi aktif secara simultan.

4. Tahap Analisis: Membandingkan urutan *output* nama kontainer yang tercetak pada layar terminal VS Code untuk menarik kesimpulan performa algoritma.

BAB IV

HASIL DAN PEMBAHASAN

A. Hasil Pengujian Sistem

Pengujian fungsionalitas sistem dilakukan dengan menguji interaksi antara skrip pengujian **tes.py**, *Load Balancer* Nginx, dan aplikasi web Flask (**app.py**) yang berjalan di dalam setiap kontainer target (web-1, web-2, dan web-3).

Ketika pengujian berjalan, alur pemrosesan data terjadi pada tiga komponen utama tersebut:

1. **tes.py (Sisi Client):** Mengeksekusi 5 *request* secara simultan menggunakan arsitektur *multi-threading* langsung ke alamat IP *Load Balancer* Nginx (<http://localhost:8085>).
2. **Nginx (Sisi Load Balancer):** Menerima paket lalu lintas data dan mendistribusikannya ke kontainer *backend* berdasarkan aturan algoritma aktif (`nginx.conf`).
3. **app.py (Sisi Aplikasi Web Backend):** Menerima *request* dialihkan dari Nginx, menahan proses selama 0.1 detik konstan (simulasi beban kerja), lalu mengirimkan kembali respons dokumen HTML berisi identitas unik masing-masing kontainer (`CONTAINER_NAME`).

Skrip `tes.py` kemudian melakukan pemfilteran (*parsing*) teks pada dokumen HTML yang diterima dari `app.py` menggunakan ekspresi reguler (*Regular Expression*), lalu mencetak hasilnya secara *real-time* pada terminal Visual Studio Code.

Berikut adalah detail hasil dokumentasi pengujian dari kedua skenario:

1. Hasil Skenario Algoritma Round Robin (Melalui Eksekusi `tes.py`)

Pada skenario pertama, pengujian dilakukan dengan menonaktifkan direktif `least_conn`; pada berkas `nginx.conf`. Ketika skrip `tesrr.py` dieksekusi di terminal, aplikasi web `app.py` menerima tugas secara bergiliran. Sistem memberikan hasil pencatatan distribusi beban sebagai berikut:

```
PS D:\Coding\2026\LC-ROUNDROBIN> python tesrr.py
--- MEMULAI PENGUJIAN ROUND ROBIN (ANTREAN SATU PER SATU) ---
Request 1: Terarah ke -> 1
Request 2: Terarah ke -> 3
Request 3: Terarah ke -> 2
Request 4: Terarah ke -> 1
Request 5: Terarah ke -> 3
PS D:\Coding\2026\LC-ROUNDROBIN>
```

Gambar 4.1: Output Eksekusi skrip tes.py pada Skenario Round Robin

Berdasarkan Gambar 4.1, ketika `tes.py` mengirimkan paket beban, Nginx mendistribusikan *request* secara kaku berdasarkan urutan baris daftar server. Karena kecepatan pemrosesan di setiap file `app.py` kontainer dibuat seragam (0.1 detik), skrip `tes.py` berhasil menangkap respons balik yang teratur dan berputar secara siklis

2. Hasil Skenario Algoritma Least Connections (Melalui Eksekusi `tes.py`)

Pada skenario kedua, berkas `nginx.conf` diperbarui dengan mengaktifkan perintah `least_conn`;, kemudian seluruh kontainer dibangun ulang (*rebuild*). Saat skrip penguji `tes.py` dijalankan kembali, log terminal memperlihatkan perubahan perilaku distribusi sebagai berikut:

```
PS D:\Coding\2026\LC-ROUNDROBIN> python tes.py
--- MEMULAI PENGUJIAN BERSAMAAN ---
Request 1: Terarah ke -> 3
Request 2: Terarah ke -> 3
Request 3: Terarah ke -> 1
Request 4: Terarah ke -> 2
Request 5: Terarah ke -> 2
```

Gambar 4.2: Output Eksekusi skrip tes.py pada Skenario Least Connections

Berdasarkan Gambar 4.2, skrip tes.py mencatat bahwa urutan respon dari aplikasi web app.py di setiap kontainer mengalami lompatan dinamis yang tidak sekuensial. Hal ini membuktikan skrip pengujian berhasil memicu logika *Least Connections* pada Nginx untuk mengalihkan lalu lintas data secara dinamis ke server yang memiliki antrian paling sedikit saat aplikasi web sedang sibuk memproses data.

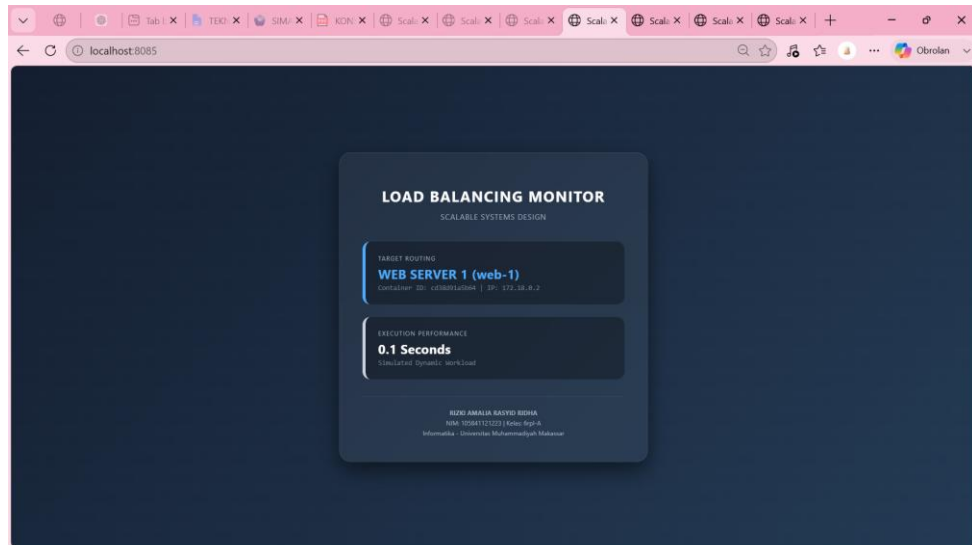
3. Tampilan Antarmuka Halaman Web (Sisi Browser Client Localhost)

Ketika pengguna melakukan pengujian manual dengan mengakses tautan *Load Balancer* melalui peramban (*web browser*) di alamat `http://localhost:8085`, aplikasi Flask (app.py) akan me-render teks respons berupa kode elemen HTML dasar.

Tampilan halaman web ini berfungsi sebagai indikator visual utama yang menunjukkan kontainer mana yang sedang aktif menangani *request* dari pengguna pada detik tersebut. Berikut adalah visualisasi tampilan antarmuka saat diakses secara langsung:

a. Tampilan Halaman Web saat diarahkan ke Web Server 1:

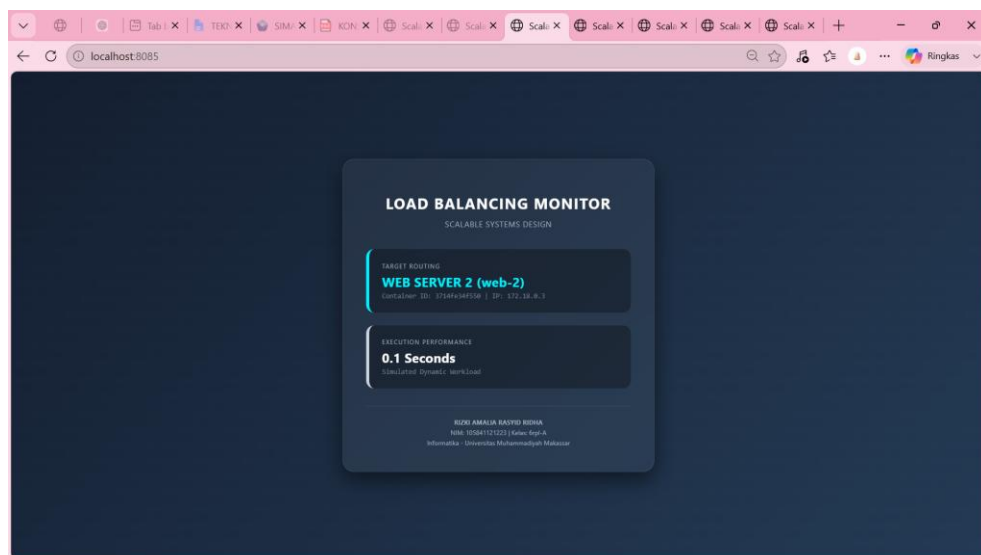
Jika Nginx mengalihkan lalu lintas data ke kontainer pertama, browser akan menampilkan teks penanda web-1 seperti pada gambar berikut:



Gambar 4.3: Tampilan Antarmuka Respon dari Kontainer web-1 pada Browser

b. Tampilan Halaman Web saat diarahkan ke Web Server 2:

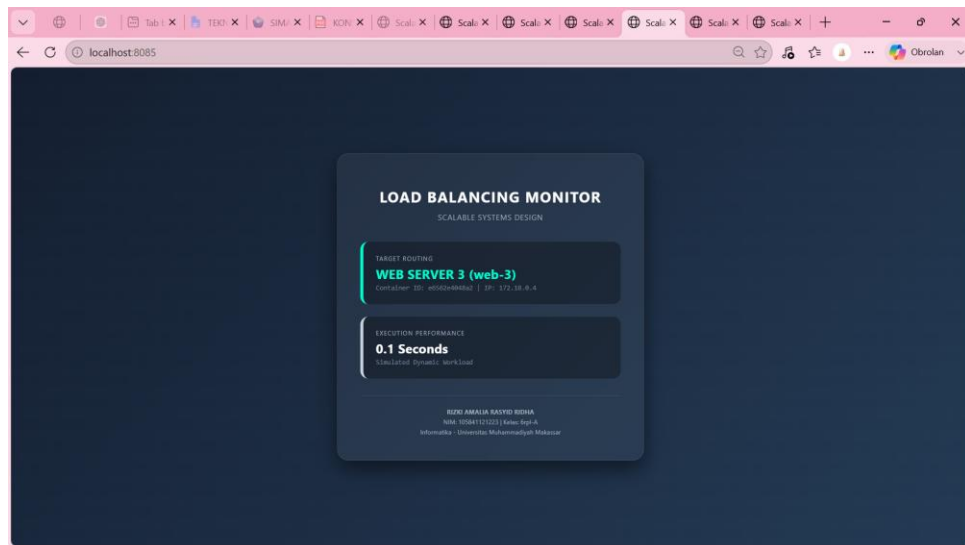
Jika Nginx mengalihkan lalu lintas data ke kontainer kedua, teks pada browser akan otomatis berubah memuat identitas kontainer web-2:



Gambar 4.4: Tampilan Antarmuka Respon dari Kontainer web-2 pada Browser

c. Tampilan Halaman Web saat diarahkan ke Web Server 3:

Begitu pula saat beban dialihkan ke kontainer ketiga, halaman browser akan menampilkan identitas kontainer web-3:



Gambar 4.5: Tampilan Antarmuka Respon dari Kontainer web-3 pada Browser

B. Analisis dan Pembahasan

1. Analisis Karakteristik Algoritma Round Robin

Berdasarkan hasil pengujian pada bagian A.1, mekanisme *Round Robin* terbukti membagi beban *traffic* menggunakan prinsip perputaran antrian (*cyclic/circular selection*). Ketika skrip pengujian mengirimkan *request*, Nginx mengalokasikan beban tersebut ke web-1, web-2, dan web-3 satu per satu sesuai dengan urutan daftar indeks *backend* yang tercatat pada *nginx.conf*.

Algoritma ini bersifat statis dan kaku (*stateless*), artinya Nginx sama sekali tidak mempertimbangkan apakah salah satu kontainer sedang mengalami lonjakan beban kerja (*workload*) atau memiliki waktu pemrosesan (*response time*) yang lebih lama akibat fungsi penundaan waktu proses di app.py. Jika sebuah *request* dilepaskan dalam jeda waktu yang teratur, polanya akan selalu berurutan tanpa ada server yang dilompati.

2. Analisis Karakteristik Algoritma Least Connections

Sebaliknya, pada bagian A.2, ketika algoritma *Least Connections* diaktifkan, terjadi perubahan pola distribusi beban yang sangat signifikan. Karakteristik utama dari algoritma ini adalah sifatnya yang dinamis dan berbasis status koneksi aktual (*stateful*).

Setiap kali paket *request* baru tiba di pintu utama Nginx, sistem *load balancer* tidak langsung melemparnya berdasarkan urutan nomor absen server. Nginx melakukan kalkulasi instan (evaluasi metrik) terlebih dahulu terhadap jumlah koneksi aktif (*active connections*) yang sedang ditahan oleh setiap kontainer Flask.

Ketika skrip menembakkan banyak *request* di milidetik yang sama, salah satu kontainer (misalkan web-1) akan langsung terisi oleh koneksi aktif yang sedang berjalan. Pada milidetik berikutnya, Nginx mendeteksi bahwa status web-1 memiliki nilai koneksi > 0, sementara web-2 dan web-3 masih bernilai 0. Secara otomatis, Nginx akan melompati antrean reguler dan langsung mengalihkan rute *request* berikutnya menuju server yang memiliki koneksi paling sedikit (paling longgar). Hal inilah yang menyebabkan urutan angka server di terminal melompat secara acak dan tidak lagi berurutan kaku seperti pada pengujian *Round Robin*.

BAB V

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian sistem *load balancing* menggunakan Nginx dan Docker yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Algoritma **Round Robin** bekerja dengan membagi beban lalu lintas data secara bergilir dan kaku sesuai urutan daftar server tanpa memedulikan status performa atau beban kerja kontainer *backend*. Algoritma ini sangat ideal diterapkan pada lingkungan server yang memiliki spesifikasi perangkat keras identik dan menangani jenis tugas yang homogen.
2. Algoritma **Least Connections** bekerja secara dinamis dengan mengalihkan lalu lintas data ke kontainer *backend* yang memiliki jumlah koneksi aktif paling sedikit pada saat itu. Algoritma ini sangat efektif untuk mencegah terjadinya penumpukan beban (*bottleneck*) ketika sistem menerima lonjakan *traffic* yang fluktuatif secara bersamaan.